

Transfer Learning for Big-5 African Wildlife Classification

Feature extraction versus fine-tuning of a pre-trained ResNet50 (TensorFlow/Keras) | Engineering report | 3 October 2026

Author: Sylvia Njane | GitHub: startechb

1. Summary

We built a five-class image classifier (buffalo, elephant, rhino, lion, leopard) on a pre-trained ResNet50 and compared two training regimes on a held-out test set of 240 images: **Experiment A** (feature extraction, backbone frozen) and **Experiment B** (fine-tuning the last residual stage at a low learning rate). Both regimes reach roughly 98% test accuracy.

Metric evaluated	Experiment A (feature extraction)	Experiment B (fine-tuning)
Best Validation Accuracy	99.16 %	99.58 %
Test Set Macro F1-Score	0.9858	0.9821
Average Epoch Training Time	15.0 sec	18.1 sec

- **Fine-tuning gave no measurable improvement.** Test accuracy was 98.33% (A, 4 errors) versus 97.92% (B, 5 errors). The models disagree on only 3 images (1 wrong only under A, 2 wrong only under B); an exact McNemar test gives $p = 1.0$. The 0.42-point validation edge for B is also a single image.
- **Cost.** B trains 15.5M parameters instead of 0.53M. Mean epoch time rose from 15.0 s to 18.1 s, but both means are inflated by a 37-42 s first epoch and, in B, by several slow epochs. The steady-state median (epochs 2-10) is 12.5 s versus 13.0 s.
- **All errors are among buffalo, elephant and rhino.** Lion and leopard are classified perfectly by both models, which should be read with care (Section 7).
- **Failure modes** are atypical presentation rather than missing capacity: a captive-enclosure photo, panoramic images squeezed to a square, crowded herds with wet dark hides, and an unusual head-on pose. Several errors are made with very high confidence.

2. Data and experimental setup

2.1 Datasets

Ultralytics African Wildlife (1,501 images; classes buffalo, elephant, rhino, zebra). The dataset is in YOLO *detection* format (images plus box labels, no class folders), and the Kaggle identifier given in the assignment (ultralytics/african-wildlife) does not resolve (HTTP 403); the official copy is the Ultralytics GitHub release, which we used. A class was derived per image from its label file. Zebra images (375) were dropped as instructed, and 18 images with boxes from more than one species or with an empty label file were excluded because they have no single correct label.

Lion and leopard come from the Kaggle dataset *10 Big Cats of the Wild* (gpiosenka/cats-in-the-wild-image-classification, CC0): the LIONS and AFRICAN LEOPARD folders from its train, valid and test parts were pooled (238 and 246 images). Snow, clouded and other leopard folders were excluded because they are different animals. Dropping zebra and adding two classes gives **five** classes, so the output layer is Dense(5) rather than the Dense(4) of the brief.

2.2 Split and class balance

All 1,595 images were pooled and split with a seeded (42) **stratified** 70/15/15 split, so every class keeps the same proportions in each partition. The test set was used only for the final evaluation.

Class	Train	Validation	Test	Total	Class weight
buffalo	259	55	56	370	0.86
elephant	260	56	56	372	0.86
leopard	172	37	37	246	1.30
lion	167	36	35	238	1.34
rhino	258	55	56	369	0.87
All	1,116	239	240	1,595	

The 1.6x imbalance between the largest and smallest classes is mild, so it was handled with class-weighted loss on the training set only. Unlike resampling, this does not duplicate images and cannot place copies of one image on both sides of the split.

2.3 Input pipeline and model

- `tf.keras.utils.image_dataset_from_directory`, 224x224 inputs, batch 32. Training images pass through `RandomFlip('horizontal')`, `RandomRotation(0.1)` and `RandomZoom(0.1)`; validation and test images are not augmented. The test set is not shuffled, so predictions align with labels.
- ResNet50 with ImageNet weights, `include_top=False`, followed by `GlobalAveragePooling2D`, `Dense(256, relu)`, `Dropout(0.5)`, `Dense(5, softmax)`. ResNet50's own `preprocess_input` is part of the model, so every split and any future inference are preprocessed identically.
- Loss: sparse categorical cross-entropy with the class weights above. After each epoch the weights with the best validation accuracy are checkpointed, and those checkpoints are what the test evaluation uses.

3. Structural changes between the two experiments

The architecture is identical in A and B. Only the set of trainable weights, the optimiser setting and the starting point change:

	Experiment A: feature extraction	Experiment B: fine-tuning
Trainable backbone layers	none (entire ResNet50 frozen)	conv5 stage (from conv5_block1_1_conv): 10 convolution layers
BatchNorm layers	frozen, inference mode	kept frozen, inference mode
Trainable parameters	525,829 of 24,113,541 (2.2%)	15,479,301 of 24,113,541 (64.2%)
Optimiser / learning rate	Adam, 1e-3	Adam, 1e-5 (re-compiled)
Epochs	10 (epochs 1-10)	10 more (epochs 11-20)
Initial weights	ImageNet backbone, random head	end of Experiment A (epoch 10)

Two design choices matter for stability. First, the backbone is always called with `training=False`, and BatchNorm layers stay frozen even in B: with about 1,100 training images, re-estimating batch statistics or scales is a common source of unstable fine-tuning. Second, the learning rate drops 100x for B so the pre-trained conv5 filters are adjusted rather than overwritten. Keras ignores a change of trainable until the model is compiled again, so B recompiles explicitly.

4. Results

4.1 Training curves across both runs

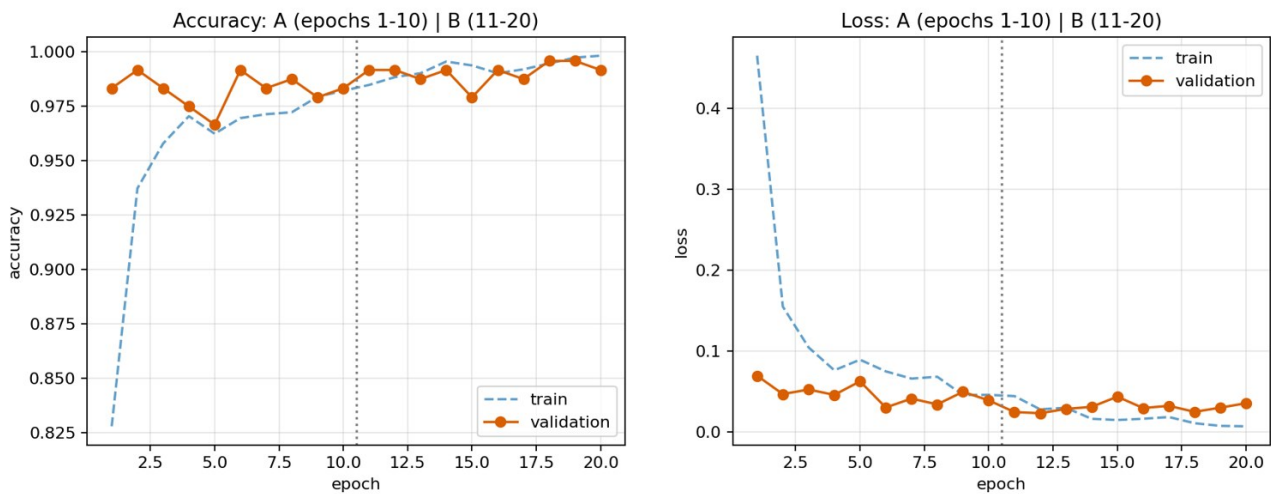


Figure 1. Training and validation accuracy and loss over 20 epochs. The dotted line marks the switch from feature extraction (A) to fine-tuning (B).

Validation accuracy is already 98.3% after the first epoch of A and then fluctuates between 96.7% and 99.2%; with 239 validation images each image is worth 0.42 points, so these swings are 2-6 images. Training accuracy is lower than validation accuracy during A because training runs with dropout and augmentation, which validation does not use. In B training accuracy climbs to 99.8% while validation loss stops improving (minimum 0.0234 at epoch 12, 0.0357 at epoch 20), the first sign of mild overfitting.

4.2 Test-set performance

Test accuracy: A 98.33% (95% Wilson interval 95.8-99.4%), B 97.92% (95.2-99.1%). The intervals overlap almost completely.

Class	Precision A	Recall A	F1 A	Precision B	Recall B	F1 B	Support
buffalo	1.000	0.982	0.991	0.982	0.982	0.982	56
elephant	0.982	0.964	0.973	1.000	0.929	0.963	56
leopard	1.000	1.000	1.000	1.000	1.000	1.000	37
lion	1.000	1.000	1.000	1.000	1.000	1.000	35
rhino	0.948	0.982	0.965	0.933	1.000	0.966	56
Macro avg	0.986	0.986	0.986	0.983	0.982	0.982	240

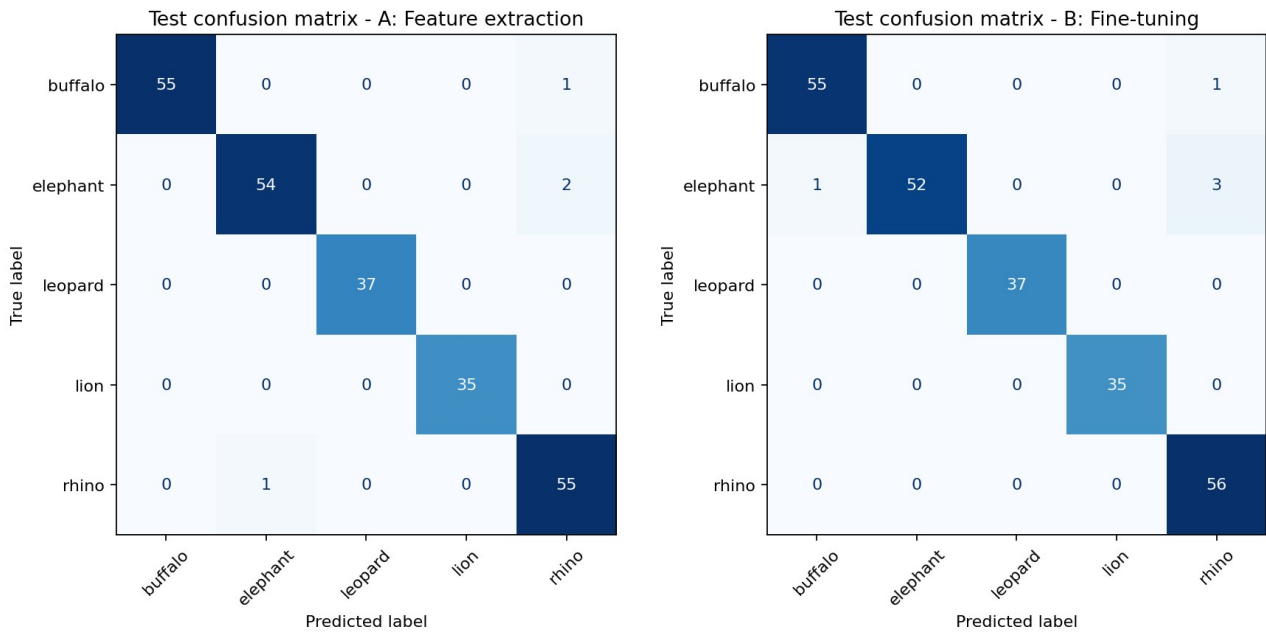


Figure 2. Test confusion matrices (rows: true class, columns: predicted). Left: Experiment A. Right: Experiment B.

5. Performance shifts

- **Is B better than A? The data do not show it.** On validation B is ahead by one image; on test A is ahead by one image. Of the 240 test images, 234 are correct under both models (3 are wrong under both). The paired exact test on the 3 disagreements (1 vs 2) gives $p = 1.0$. One run per setting means we also cannot separate these differences from training-seed noise.
- **Why so little headroom?** Frozen ImageNet features already separate these five species almost perfectly (A reaches 98.3% validation accuracy after one epoch), and the dataset is small. Fine-tuning has little left to gain and a higher capacity to memorise 1,116 training images; B's training accuracy rises to 99.8% without a matching validation gain.
- **Where the models differ.** B moves errors toward elephant-to-rhino: elephant recall falls from 0.964 to 0.929 and rhino precision from 0.948 to 0.933, while rhino recall rises to 1.000. B is also more confident overall (mean confidence on correct predictions 0.996 vs 0.971), but its mistakes are as confident as A's (0.85 vs 0.81 mean).
- **Compute.** The reported means (A 15.0 s, B 18.1 s) include a first epoch of 37 s and 42 s (GPU warm-up and cache filling) and three B epochs of about 20 s that are probably shared-GPU contention. Excluding the first epoch, the medians are 12.5 s and 13.0 s, so the true per-epoch overhead of back-propagating through conv5 on a T4 is small, although B also needs the extra epochs and one more compile step. We report the required means in the results matrix and the medians here for honesty.
- **Operational conclusion.** For a dataset of this size and difficulty, feature extraction is the better engineering choice: it trains 29x fewer parameters, is simpler to maintain, and matches B on every metric we can resolve.

6. Error analysis under localised edge cases

Across both experiments six distinct test images are misclassified (4 under A, 5 under B, three under both). All are Ultralytics-source images of buffalo, elephant or rhino. The table lists each failure, the visible condition, and a *hypothesis* for the cause. The hypotheses are inferred from the images; we did not run controlled tests to confirm them.

Image	True / predicted (confidence)	A	B	Edge case visible in the image and likely cause
wild_2 (54)	elephant / rhino (0.99 A, 1.00 B)	wrong	wrong	Captive enclosure with concrete walls, unusual camera angle, mud-grey smooth hide, second animal partly hidden. Out-of-distribution background; texture and colour resemble rhino.
wild_2 (299)	elephant / rhino (0.84 A, 0.98 B)	wrong	wrong	Very wide panorama with small animals. The resize to 224x224 squeezes the image horizontally, distorting body shape, and the subjects occupy few pixels.
wild_1 (136)	buffalo / rhino (0.79 A, 0.73 B)	wrong	wrong	Single dark, bulky animal in side view among shrubs: a pose and body mass that buffalo and rhino share. The only low-confidence error repeated by both models.
wild_2 (317)	elephant / rhino (0.75 B)	right	wrong	Dry grey-brown savanna, second elephant partly occluded, wide aspect ratio (also squeezed). Fine-tuning moved this image across the boundary.
wild_2 (70)	elephant / buffalo (0.79 B)	right	wrong	Crowded herd bathing in a river, wet dark hides, partial occlusion, people and harnesses. One label for a multi-animal scene.
wild_3 (113)	rhino / elephant (0.64 A)	wrong	right	Rhino charging head-on with dust and a second rhino behind: an uncommon pose, giving a low-confidence error that fine-tuning corrected.

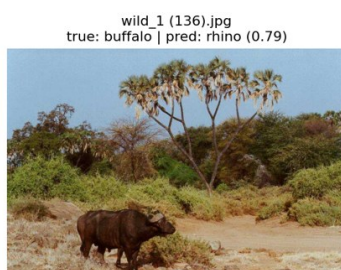


Figure 3. Misclassified test images, Experiment A (true label, predicted label and confidence in each title).

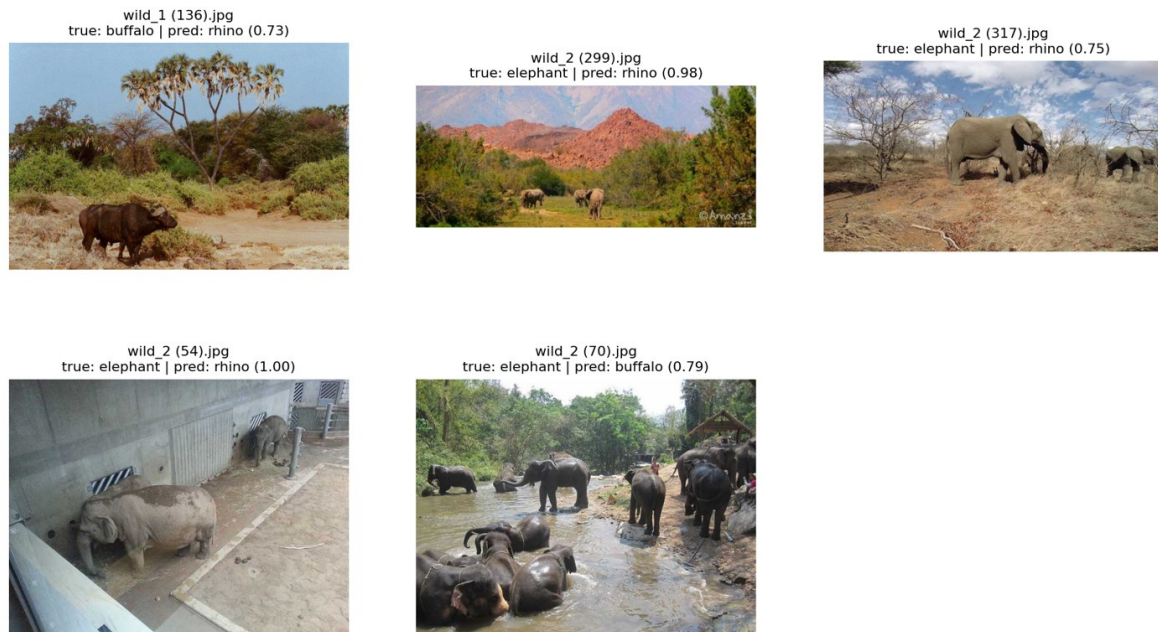


Figure 4. Misclassified test images, Experiment B.

What the failures have in common

- **Direction.** Four of B's five errors are elephants labelled rhino or buffalo, and rhino is the most frequent wrong answer. The model confuses large, grey or dark, thick-skinned quadrupeds, not the cats.
- **Presentation, not class.** Each failure involves something outside the typical training example: enclosure background, panoramic framing, wet herds, partial occlusion or an unusual pose.
- **Confidence is not a safe filter.** The enclosure photo is wrong at 0.99-1.00 confidence in both models, so a simple confidence threshold would not catch it. Mean confidence on errors is 0.81 (A) and 0.85 (B).
- **Label granularity.** The source is a detection dataset: several test images contain many animals, and a single image-level label is a coarse description of them.

Suggested mitigations (not yet tested)

- Preserve aspect ratio when resizing (`crop_to_aspect_ratio=True` or `padding`) so panoramic photos are not distorted.
- Add training images of captive settings, herds, partial views and unusual poses, and add contrast/colour augmentation.
- Repeat each experiment over several seeds, or use cross-validation, to measure the A-versus-B difference with a proper confidence interval.
- For herd scenes, switch to detection or multi-label outputs instead of one label per image.

7. Limitations

- **Small test set.** 240 images give a 95% interval of about 4 points on 98% accuracy, and only 4-5 errors drive every difference reported.
- **Single run per experiment** (seed 42); training-seed variance was not measured. "Best validation accuracy" is a maximum over epochs, so it is slightly optimistic.
- **Perfect lion and leopard scores may reflect the data source.** These classes come from a different dataset with its own photographic style, and cats look very different from the other three species, so part of the separation could be a dataset shortcut. We did not test this, for example by evaluating on lion or leopard

photos from another source.

- **No near-duplicate check.** We did not check for near-duplicate photos across the train/test boundary; if present they would inflate every score.
- **One fine-tuning configuration.** Only conv5 at $1e-5$ for 10 epochs was tried. The result shows that this setting gave no gain, not that fine-tuning never helps. Experiment B also started from the final epoch of A, not from A's best checkpoint, and A's reported test result uses its best-validation checkpoint (early in training, epoch 2).

8. Conclusion

A frozen ResNet50 with a small trained head classifies the five Big-5 species at about 98% accuracy on unseen images, and unfreezing the last residual stage changed nothing we can measure while multiplying the trainable parameters by 29. The remaining errors are concentrated in buffalo, elephant and rhino images with atypical framing, backgrounds or poses, which points to data coverage and preprocessing, not to model capacity, as the next things to improve.

Appendix A. Per-epoch training log

Epoch	Phase	Train acc	Val acc	Val loss	Time (s)
1	A	0.8280	0.9833	0.0697	37.1
2	A	0.9373	0.9916	0.0470	12.9
3	A	0.9579	0.9833	0.0527	12.7
4	A	0.9704	0.9749	0.0460	12.5
5	A	0.9624	0.9665	0.0628	12.5
6	A	0.9695	0.9916	0.0305	12.6
7	A	0.9713	0.9833	0.0415	12.4
8	A	0.9722	0.9874	0.0343	12.6
9	A	0.9794	0.9791	0.0499	12.2
10	A	0.9821	0.9833	0.0393	12.0
11	B	0.9848	0.9916	0.0248	42.3
12	B	0.9884	0.9916	0.0234	13.1
13	B	0.9901	0.9874	0.0287	20.2
14	B	0.9955	0.9916	0.0313	12.0
15	B	0.9937	0.9791	0.0438	21.8
16	B	0.9901	0.9916	0.0298	20.2
17	B	0.9919	0.9874	0.0325	12.9
18	B	0.9946	0.9958	0.0251	13.0
19	B	0.9973	0.9958	0.0303	12.9
20	B	0.9982	0.9916	0.0357	12.9

Appendix B. Reproducibility

- Environment: Google Colab, Tesla T4 GPU, TensorFlow/Keras. Seed 42 for the train/validation/test split and for the Python, NumPy and TensorFlow random generators.
- Code: `notebooks/African_Wildlife_Transfer_Learning.ipynb` in the project repository (Startechb/Transfer-Learning-on-the-Ultralytics-African-Wildlife). The notebook downloads both datasets, builds the five-class folder, runs both experiments and produces every figure and table in this report.
- Data sources: Ultralytics African Wildlife (GitHub release v0.0.0 of `ultralytics/assets`) and Kaggle `gpiosenka/cats-in-the-wild-image-classification`.